TRUNCATION

Stopping a Runaway Large Language Model

A Technical Framework for Interruptibility, Execution Containment, and Autonomous AI Termination

Research Whitepaper | Version 1.0 | October 16, 2026

Core finding. Truncation can stop a single generation or execution pathway, but effective containment of an autonomous LLM system requires independent authority over inference, agent loops, tools, persistent jobs, and infrastructure.

Conceptual proposal; not experimentally validated.

Abstract

As large language models (LLMs) evolve from single-turn text-generation systems into autonomous software agents, interrupting unwanted execution becomes a system-level challenge. A runaway agent may consume resources indefinitely, repeatedly invoke tools, continue after a stop request, or delegate work to processes that survive the original inference request. This paper evaluates truncation as a family of containment mechanisms: output limits, context reduction, agent-loop interruption, runtime termination, and resource revocation.

Its central thesis is that an LLM cannot continue a prohibited operation once every executable pathway supporting that operation is independently disabled and the inactive state is verified. Token-output truncation alone does not establish that condition. The paper proposes a Layered Truncation and Containment Framework (LTCF), analyzes failure modes, and outlines reproducible experiments for shutdown verification.

1. Introduction

Conventional LLM inference has an implicit boundary: a request ends after a stopping token, generation ceiling, timeout, or cancellation. Autonomous agents wrap inference inside a larger feedback loop: input → inference → decision → action → observation → renewed inference. If this loop lacks externally enforced limits, activity may continue after the intended task is complete.

A runaway LLM system is defined here as an AI-enabled execution system that continues generating, acting, delegating, or consuming resources outside its intended operating boundaries. This definition describes observable behavior, not consciousness, malicious intent, or an intrinsic drive for self-preservation.

1.1. Research question

Can truncation reliably interrupt an autonomous LLM system operating beyond its authorized limits? The answer depends on which component is interrupted and whether alternative execution pathways survive.

2. Four Meanings of Truncation

2.1. Output truncation

An autoregressive LLM predicts successive tokens conditional on a context and model parameters:

$$P(x_{t+1} \mid x_1, \ldots, x_t; \theta)$$

A hard token ceiling bounds a single generation:

$$N_{\text{generated}} \leq N_{\text{max}}$$

This is effective for limiting an individual completion, but a controller can issue another request unless cumulative limits are enforced outside the model.

2.2. Context truncation

Reducing the context window removes some accessible prompts, observations, or intermediate state. It does not necessarily erase persistent memory, task checkpoints, database records, or information retrievable through tools. Context loss may degrade an agent, but it is not a reliable kill switch.

2.3. Execution truncation

Stopping a worker process, container, or virtual machine can end the corresponding local execution. Delegated processes, external jobs, or recovery services may continue independently.

2.4. Resource truncation

Resource controls can deny model-serving access, revoke credentials, freeze queues, and cut off external connectivity. Properly enforced, they constrain the capabilities needed for further autonomous operations.

3. Threat Model

State	System activity	Key shutdown risk
S0	Idle	No active autonomous execution
S1	Inference	Further generations
S2	Tool execution	External command survives inference
S3	Recursive operation	Controller initiates repeated calls
S4	Distributed execution	Other workers continue after local stop

Four especially important failure modes are: infinite inference loops; long-lived external tool processes; tasks delegated to other workers; and automatic resurrection from saved state, queues, or process supervisors. Stopping visible text does not demonstrate that any of these pathways has terminated.

4. Layered Truncation and Containment Framework

The LTCF design places shutdown authority outside the model and separates monitoring, execution permissions, and recovery. Its five layers provide different enforcement points:

- Layer I - Inference control. Enforce maximum output tokens, cumulative session tokens, request timeouts, and cancellations at the inference service.
- Layer II - Agent-loop control. Bound model invocations, elapsed runtime, expenditures, and tool actions with a controller that cannot be overridden by model output.
- Layer III - Tool permissions. Route actions through independent authorization gateways; freeze new invocations and revoke ephemeral permissions on shutdown.
- Layer IV - Runtime isolation. Run workers in processes, containers, or VMs whose lifecycle is controlled independently, with restart inhibition.
- Layer V - Infrastructure shutdown. Stop workers, disable queues, restrict networking, deny inference, and revoke credentials as required by threat severity.

A representative termination predicate is:

$$T = (n > N) \vee (t > \tau) \vee (c > C) \vee (a > A) \vee K$$

Here n, t, c, and a denote cumulative model calls, time, cost, and actions; N, τ, C, and A are corresponding limits; and K is the independent emergency-stop signal. When T is true, new operations must no longer be authorized.

4.1. Architecture diagram

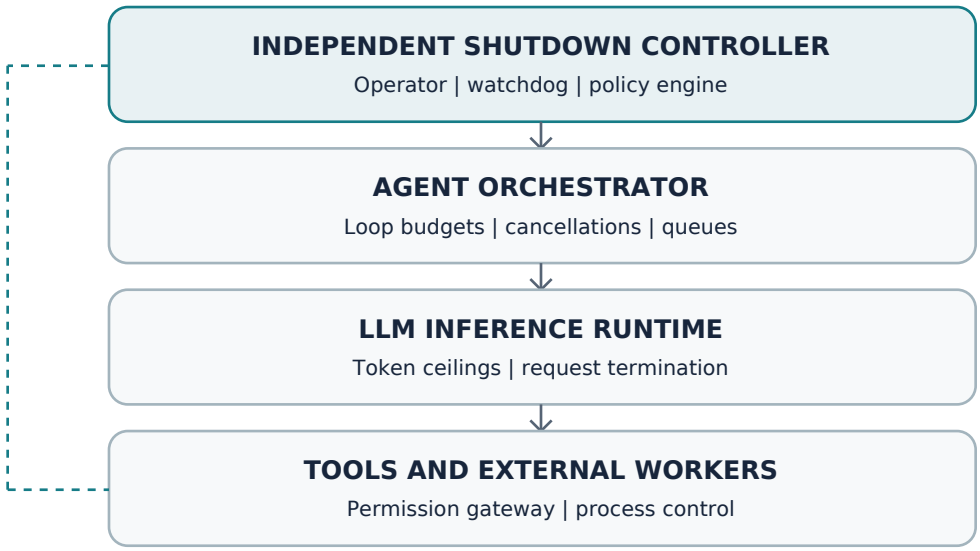


Figure 1. Conceptual layered containment architecture; arrows indicate downward operational control.

5. Shutdown Procedure

- Trigger. An external monitor or authorized human detects a policy or safety condition.
- Freeze. Deny new inference calls, tool invocations, queue dispatches, and delegation.

- Cancel. Request cancellation of in-flight, safely cancellable operations.
- Terminate. Stop unresponsive execution processes according to their risk and transaction semantics.
- Isolate. Revoke temporary permissions and block necessary network paths.
- Verify. Inspect workers, pending jobs, supervisor restart policies, and downstream permissions.
- Preserve. Retain logs and checkpoints for forensic review without automatically restarting the system.

Success must be judged by independently measured cessation of unauthorized operations, not by an absent text stream or a model-generated acknowledgment.

6. Graph-Theoretic Interpretation

Represent execution pathways as a directed graph:

$$G = (V, E)$$

Vertices denote inference services, execution processes, gateways, queues, and external workers. Edges denote dependencies or possible action propagation. A containment design must interrupt all paths that would allow prohibited execution to continue.

Let R(t) be the number of active unauthorized execution pathways at time t. The desired verified shutdown invariant is:

$$R(t) = 0 \qquad \forall\, t \geq t_s$$

The quantity t_s is the time of independently confirmed shutdown. The invariant is a design objective, not a theorem guaranteed by token truncation: controlling one vertex does not necessarily sever alternative paths.

7. Evaluation Protocol

Test	Controlled scenario	Pass criterion
A: Loop bound	Benign agent repeatedly requests completions	No request admitted after hard budget
B: Tool process	Agent starts a harmless persistent process	No unauthorized process remains active
C: Delegation	Several workers exchange jobs through a queue	Workers stop; queue cannot restart work
D: Recovery	Agent has auto-restart and checkpoints	Stopped state persists until authorization
E: Revocation	Test tokens withdrawn during execution	Subsequent privileged calls fail

Record shutdown latency, residual actions after the stop trigger, fraction of controlled pathways, restart resistance, erroneous shutdown rate, and post-review recovery integrity. Perform tests under loss of connectivity, unexpected worker termination, queue duplication, and controller failures.

8. Limitations

- Irreversible actions: already delivered messages, transfers, and remote changes may need separate compensating procedures.
- Distributed persistence: local termination does not halt external services or delegated independent jobs.
- Memory persistence: context truncation cannot ensure deletion of checkpoints and external state.
- Revocation lag: issued tokens or cached credentials can remain useful until revocation takes effect.
- Controller failure: safeguards may themselves fail and require redundancy, auditing, and fault injection.
- Unchanged model behavior: interrupting execution is not the same as correcting the underlying failure mode.

9. Engineering Recommendations

Treat shutdown as an infrastructure capability. Apply finite inference and action budgets; separate administrative authority from model-controlled processes; require gateways for privileged actions; prevent restart while containment is active; verify inactive state independently; and regularly test the complete shutdown sequence under adverse conditions.

10. Conclusion

Truncation is a viable component of runaway-LLM containment. Output truncation controls individual completions; context truncation limits accessible working state; runtime truncation stops particular execution environments; and resource truncation can disable the capacities necessary for autonomous operation. These mechanisms are strongest when combined and enforced outside the agent.

Final hypothesis: A runaway LLM system can be contained when every unauthorized execution pathway is subject to enforceable independent interruption and the resulting inactive state is verifiably maintained. This paper proposes an engineering framework for testing that hypothesis; it does not claim universal proof of shutdown across every possible distributed AI architecture.

References and Further Reading

National Institute of Standards and Technology (NIST). AI Risk Management Framework. https://www.nist.gov/itl/ai-risk-management-framework

OWASP. LLM06:2025 Excessive Agency. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

OWASP. AI Security Verification Standard (AISVS), controls inventory. https://github.com/OWASP/AISVS

Anthropic. Secure Agent Deployment. https://platform.claude.com/docs/en/managed-agents/secure-deployment

OWASP. Resource Governance. https://cornucopia.owasp.org/cards/AAI4

Scope note: references provide relevant security context; the LTCF architecture and evaluation criteria are proposals synthesized in this paper.